

Markov Decision Process and The Bellman Equation

CSC 480: Artificial Intelligence, Spring 2026

Guest lecture, Samuel Kaplan: Course Notes

Abstract

I Markov Decision Process

How do we plan when the world is filled with uncertainty? Moreover, how can we plan when we might end up right back where we started?

Our earlier search algorithms found optimal paths through deterministic acyclic state spaces, and expectimax extended this to stochastic outcomes. But both approaches plan from a *single starting state* outward through a tree of possible futures. Neither can appropriately handle:

- States that *cycle* (the agent may return to a state it has already visited)
- Decisions that must be made from *any* state the agent might find itself in
- Agents acting continuously over time, without a clear terminal goal

Previously we always had to plan far ahead, searching through a tree of possible futures to find the best outcome. But what happens when we need to adapt as we go, in situations that are cyclic or ongoing? This is where Markov Decision Processes come in.

A totally unrelated example

Suppose it's the night before a given lecture, and our unassuming lecturer was prepping the notes. Our lecturer's goal is to work hard and finish this lecture on time. To help our lecturer out, let's describe this world in three states the lecturer can be in:

- Writing: Actually making progress on the notes.
- Googling: Looking something up "really quickly."
- Doomscrolling: Completely off task, watching videos about whatever the algorithm has decided to serve up tonight.

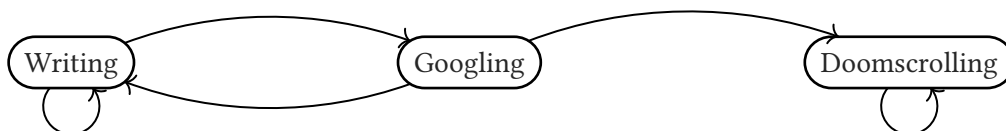


Figure 1: State Transition Diagram for Lecture Preparation

The lecturer has exactly two actions available in both Writing and Googling: *Focus* (the safe, steady option) and *Browse* (the tempting but risky one). From Writing, Focus always keeps you writing. Browse is a coin flip: half the time you find something useful and stay productive, half the time you end up down

Contents

I	Markov Decision Process	1
	A totally unrelated example	1
	The Markov Property	3
	Policy	3
	Horizons	4
	Discounting	4
II	The Bellman Equation	7
	Value and Action-Value	7
	The Bellman Equation	7
	Expectimax	8
	Applying to the Example	9

a Googling rabbit hole. From Googling, Focus gives you even odds of snapping back to Writing or staying stuck. But choosing Browse from Googling? That sends you straight to Doomscrolling, every single time.

A Markov Decision Process gives us the formal language to describe exactly this kind of problem.

Definition: Markov Decision Process

A Markov Decision Process (MDP) consists of:

- A set of *states* S
- A set of *actions* $A(s)$ available in state $s \in S$
- A *transition model* $T(s, a, s') = P(s' \mid s, a)$: the probability of landing in state s' after taking action a in state s
- A *reward function* $R(s, a, s')$ giving the immediate reward for each transition ¹
- A *discount factor* $\gamma \in [0, 1]$ controlling how much future rewards are valued relative to immediate ones ²

Note that for each state-action pair (s, a) , the transition probabilities must form a valid probability distribution:

$$\forall s, a : \sum_{s'} T(s, a, s') = 1 \quad (1)$$

For now let's ignore the discount factor and explore our example, the full transition and reward model is:

State	Action	$P'(\text{Writing})$	$P'(\text{Googling})$	$P'(\text{DS})$	Reward
Writing	Focus	1.0	0.0	0.0	+1
Writing	Browse	0.5	0.5	0.0	+2
Googling	Focus	0.5	0.5	0.0	+1
Googling	Browse	0.0	0.0	1.0	-10

Table 1: Transition model $T(s, a, s')$ and rewards for Lecture Preparation

Focus always keeps you on task, from either Writing or Googling it never sends you somewhere worse:

$$T(\text{Writing}, \text{Focus}, \text{Writing}) = 1.0$$

$$T(\text{Googling}, \text{Focus}, \text{Writing}) = 0.5, \quad T(\text{Googling}, \text{Focus}, \text{Googling}) = 0.5$$

Focus yields a modest +1 per step. Browse is the risky choice, from Writing it is a coin flip between staying productive and sliding into Googling, and from Googling it sends you straight to Doomscrolling with certainty:

$$T(\text{Writing}, \text{Browse}, \text{Writing}) = 0.5, \quad T(\text{Writing}, \text{Browse}, \text{Googling}) = 0.5$$

$$T(\text{Googling}, \text{Browse}, \text{DS}) = 1.0$$

¹The reward function is sometimes simplified as $R(s)$ or $R(s, a)$ depending on the problem. The full form $R(s, a, s')$ is the most general.

²The definition writes $\gamma \in [0, 1]$ but finite and infinite horizon problems have different requirements. For infinite horizon problems $\gamma = 1$ is not allowed since the sum may diverge, so in practice $\gamma \in [0, 1)$.

Browse yields +2 from Writing when it goes well, but the catastrophic $R(\text{Googling}, \text{Browse}, \text{DS}) = -10$ makes it a serious gamble from Googling.

Updating the diagram with rewards on each state and transition probabilities labeled by action:

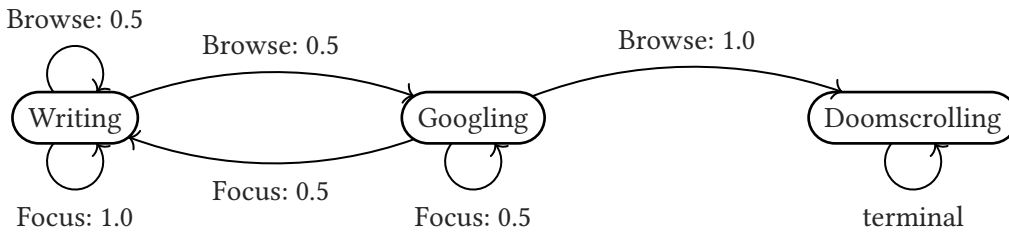


Figure 2: State Transition Diagram with Rewards and Transition Probabilities

The Markov Property

The defining feature of a Markov Decision Process is the *Markov property*, in that the future depends only on the *present state*, not on the history of how the agent arrived there.

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = P(s_{t+1} \mid s_t, a_t) \quad (2)$$

The current state is a sufficient summary of all relevant history. This is the same assumption underlying the Markov Blanket from Bayes Networks, and it is what makes MDPs tractable. Rather than reasoning about arbitrarily long decision histories, we only need to reason about where we are right now.

In our example, whether our poor lecturer spirals from Googling into Doomscrolling depends only on being in the Googling state at that exact moment, not on how many times they have previously cycled between Writing and Googling, or how many hours the deadline has been looming.

Policy

In a search problem, the solution is a *sequence of actions*. In an MDP, since the environment is stochastic and the agent may find itself in any state, the solution is instead a *policy*.

Definition: Policy

A *policy* $\pi : S \rightarrow A$ is a mapping from states to actions. The *optimal policy* π^* maximizes the expected total discounted reward from any starting state.³

The important difference from search is that the solution to an MDP is a *complete plan for every state*, not just from a single start. Wherever the environment drops us (whether we stayed focused, got distracted, or fell headfirst into a Doomscrolling spiral), a policy tells us what to do next.

³In modern reinforcement learning, finding π^* is exactly what systems like AlphaGo and RLHF tuned language models are doing, the “policy” is literally the neural network being trained.

Discussion

What is the optimal policy π^* for this MDP? Is Browse from Googling ever a rational choice given the current model? What would need to change, in the rewards or transition probabilities for it to become so?

The optimal policy is to Focus in both Writing and Googling. From Writing, Browse earns a higher immediate reward of +2 but risks landing in Googling, from which Browse sends you deterministically into Doomscrolling (a state with no exit and a reward of -10 every step for the rest of time). Browse from Googling is therefore *never* rational under this model as no immediate reward, however large, can outweigh the infinite stream of -10 penalties that follows. For Browse from Googling to become rational, the model would need to give Doomscrolling a way out (some non-zero $T(\text{DS}, \cdot, \text{Writing})$), or make the reward from Doomscrolling positive.

Horizons

A key structural choice in any MDP is whether the agent acts for a fixed number of steps or indefinitely.

In a *finite horizon* MDP with horizon T , the agent takes exactly T actions. The optimal policy here can be *non-stationary* in that the right choice at step $t = T - 1$ may differ from the right choice at step $t = 1$, because the agent has less time left to recover from a bad outcome. Our example with a lecture deadline the next day is an example of a finite horizon problem.

In an *infinite horizon* MDP, the agent acts indefinitely with no fixed endpoint. Here the optimal policy is *stationary* in that the same action is optimal in a given state regardless of how many steps have already elapsed, since there is no “time remaining” to track. This is a dramatic simplification. The Bellman equation, which we will get to shortly, applies to the infinite horizon setting.

Discounting

Infinite horizon MDPs introduce a new problem in that, if the agent collects rewards forever, the sum $\sum_{t=0}^{\infty} R_t$ may diverge. Consider our humble lecturer, if they Focus in Writing indefinitely, collecting +1 every step, the undiscounted total is $1 + 1 + 1 + \dots = +\infty$. If they instead end up Doomscrolling, the total is $(-10) + (-10) + (-10) + \dots = -\infty$. Both diverge, and comparing $+\infty$ to $-\infty$ to decide which policy is better is not meaningful. We cannot say the writing policy is “ $+\infty - (-\infty)$ better” than doomscrolling, since that arith-

metric is undefined. The solution is the *discount factor* $\gamma \in [0, 1)$ that we set aside from the MDP definition. Instead of summing raw rewards, the agent maximizes the total *discounted* reward:

$$\sum_{t=0}^{\infty} \gamma^t R_t \quad (3)$$

Since rewards are bounded (say $|R_t| \leq R_{\max}$), this geometric series converges to at most $\frac{R_{\max}}{1-\gamma}$, so values are always finite and well-defined.

There are two useful ways to think about γ . First, as a *preference for sooner rewards*, a reward of +1 now is worth more than a reward of +1 next step, just as a dollar today is worth more than a dollar next year. Second, as a model of *uncertain termination*. Imagine that after each step, a coin is flipped, with probability $1 - \gamma$ the game simply ends and the agent collects no further reward, and with probability γ it carries on. The reward at step t is then only collected if the agent survives all t preceding flips, which happens with probability γ^t (the factor multiplying that reward in the discounted sum).

The parameter γ directly controls the agent's patience:

- $\gamma = 0$: completely myopic, only the immediate reward matters, future consequences are ignored entirely. Think your average gen α gremlin hedonistically seeking rewards.
- $\gamma \rightarrow 1$: increasingly patient, distant future rewards count nearly as much as immediate ones. Think of the lecturer who refuses to Browse from Googling precisely because they know exactly where it leads and they feel the full weight of infinite future procrastination bearing down on each decision.

In our example, the value of being stuck in Doomscrolling forever is:

$$V(\text{DS}) = \sum_{t=0}^{\infty} \gamma^t (-10) = \frac{-10}{1-\gamma} \quad (4)$$

As $\gamma \rightarrow 1$ this diverges to $-\infty$, which is precisely why $\gamma < 1$ is required for infinite horizon problems, without it, values are undefined and no meaningful policy comparison is possible.

Discussion

1. When $\gamma = 0$, what is the optimal action from Writing? From Googling? Now consider any $\gamma \in (0, 1)$: is there a value of γ for which Browse from Writing is *not* optimal?
2. Let $V(s)$ denote the total expected discounted reward from state s under the optimal policy. Write $V(\text{Writing})$ as an equation involving $V(\text{Writing})$ and $V(\text{Googling})$, then do the same for $V(\text{Googling})$. Do you notice anything interesting about these equations?

When $\gamma = 0$ the agent ignores all future consequences, so from Writing it simply takes whichever action yields the highest immediate reward: Browse (+2) beats Focus (+1). From Googling, Browse yields -10 and Focus yields $+1$, so Focus is the clear choice. The optimal policy when $\gamma = 0$ is therefore Browse from Writing and Focus from Googling.

Perhaps surprisingly, Browse from Writing remains optimal for *every* $\gamma \in [0, 1)$, as there is no threshold at which Focus becomes preferable. The reason is that Googling under Focus is not a bad state to be in is that it yields $+1$ per step and returns to Writing with probability 0.5 each step. The extra $+1$ immediate reward from Browse always outweighs the modest cost of occasionally landing in Googling rather than staying in Writing. Browse from Googling, by contrast, is never rational for any $\gamma > 0$, since it leads with certainty to Doomscrolling, a state whose value $-10/(1 - \gamma)$ is catastrophically negative for any reasonable γ .

Under the optimal policy ($\pi^*(\text{Writing}) = \text{Browse}$, $\pi^*(\text{Googling}) = \text{Focus}$), the values satisfy:

$$\begin{aligned} V(\text{Writing}) &= 2 + \gamma(0.5 \cdot V(\text{Writing}) + 0.5 \cdot V(\text{Googling})) \\ V(\text{Googling}) &= 1 + \gamma(0.5 \cdot V(\text{Writing}) + 0.5 \cdot V(\text{Googling})) \end{aligned} \quad (5)$$

These are two simultaneous linear equations in two unknowns, and they can be solved directly. These equations make each state equal the immediate reward of the optimal action plus the discounted expected value of the states it leads to. This recursive structure, with value defined in terms of other values, is the core concept behind *dynamic programming*⁴. Rather than searching over the exponentially many possible action sequences, dynamic programming breaks the problem into overlapping subproblems and once we know the value of every state, we know the optimal action from every state, and those values can be computed by solving the system of equations above. Generalizing it and using it to compute optimal policies is exactly what we will do next.

⁴Richard Bellman, who lends his name to the next section, coined the term “dynamic programming” deliberately to obscure the mathematical research he was doing. https://en.wikipedia.org/wiki/Dynamic_programming#History_of_the_name

II The Bellman Equation

In the previous section we wrote down the value equations for Writing and Googling under a specific policy. But there was something unsatisfying about that derivation, we had to know the optimal policy first (Browse from Writing, Focus from Googling) before we could write the equations. In general we do not know the optimal policy, that is precisely what we are trying to find. The Bellman equation gives us a way to express the optimal values without assuming we already know the answer.

Value and Action-Value

We define $V^*(s)$ as the *optimal value* of state s , the expected total discounted reward an agent collects starting from s and acting optimally from that point forward. This is the quantity we want to compute.

Now, the value of a state depends entirely on what the agent does there. The value of taking a specific action a from state s , and then acting optimally afterward, is called the *action value* or *Q-value*:

Definition: Q-value

$$Q^*(s, a) = \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')] \quad (6)$$

The Q-value $Q^*(s, a)$ is the expected discounted reward of taking action a in state s and then following the optimal policy from every subsequent state.

The Q-value is the *expected value* of $(R(s, a, s') + \gamma V^*(s'))$ over all possible next states s' . The transition probabilities $T(s, a, s')$ sum to 1 over all s' as they are the weights of a probability distribution. Since the agent cannot control which s' it actually lands in, the Q-value must weigh each possible outcome by how likely it is.

The optimal value of a state is then simply the Q-value of the best available action:

$$V^*(s) = \max_a Q^*(s, a) \quad (7)$$

And the optimal policy just picks that best action:

$$\pi^*(s) = \arg \max_a Q^*(s, a) \quad (8)$$

The Bellman Equation

Substituting the definition of Q^* into $V^*(s) = \max_a Q^*(s, a)$ gives the *Bellman equation*:

Definition: Bellman Equation

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') [R(s, a, s') + \gamma V^*(s')] \quad (9)$$

Read this equation carefully. The left-hand side is the value of state s . The right-hand side picks the best action (the $\max_a$), then takes the expectation over where that action leads (the $\sum_{s'} T(s, a, s')$), collecting the immediate reward R and the discounted value of the next state $\gamma V^*(s')$. V^* appears on both sides as the equation is recursive.

This might look circular, but it is not a contradiction. For a finite state space with $\gamma < 1$, the Bellman equation form a system with a unique solution. The challenge is not existence but computation, how do we actually solve for V^* ?

Expectimax

The structure of the Bellman equation should look familiar. The $\max_a$ is an agent node; the agent picks the action that maximizes its value. The $\sum_{s'} T(s, a, s') [\dots]$ is a chance node; the environment picks the next state according to the transition probabilities. This is the expectimax tree from earlier in the course, but written as a set of equations over all states simultaneously rather than a tree expanded from a single root. The key difference is that the Bellman equation handles cycles: because we write equations for every state at once, we do not need to expand the tree and risk infinite loops.

The circle nodes in the tree below are *Q-states*, with each one representing a specific (s, a) pair, the moment after the agent has committed to an action but before the environment has revealed where it lands. This is the same structure as the chance nodes from expectimax, as the agent controls the edge going into a Q-state (by choosing an action), and the environment controls the edges going out of it (by sampling s' according to $T(s, a, X)$). Agent nodes take a max over their children just as in expectimax, and Q-states take the expectation over theirs.

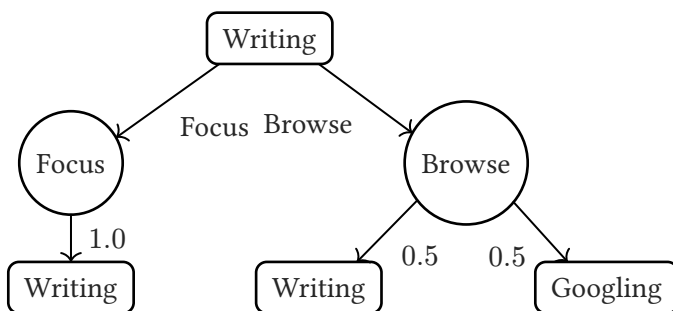


Figure 3: Expectimax trees for Writing. Square nodes are agent (max) nodes, circle nodes are Q-states where the environment resolves the outcome.

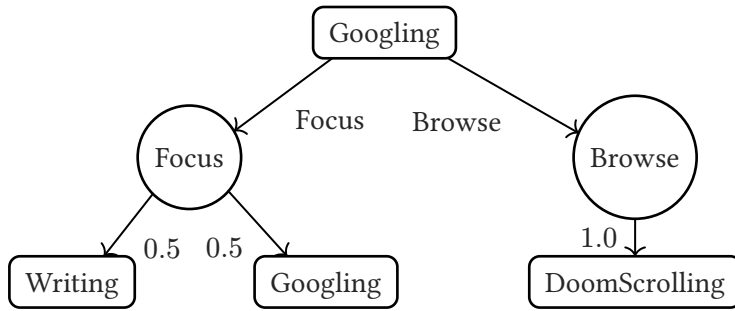


Figure 4: Expectimax tree for Googling.

Applying to the Example

Expanding the Bellman equation directly for each non-terminal state:

$$V^*(\text{Writing}) = \max \begin{cases} 1 + \gamma V^*(\text{Writing}) & [\text{Focus}] \\ 2 + \gamma(0.5 \cdot V^*(\text{Writing}) + 0.5 \cdot V^*(\text{Googling})) & [\text{Browse}] \end{cases}$$

$$V^*(\text{Googling}) = \max \begin{cases} 1 + \gamma(0.5 \cdot V^*(\text{Writing}) + 0.5 \cdot V^*(\text{Googling})) & [\text{Focus}] \\ -10 + \gamma V^*(\text{DS}) & [\text{Browse}] \end{cases}$$

We already know Browse dominates from Writing and Focus dominates from Googling for all $\gamma \in [0, 1)$, so the max resolves and we recover exactly the simultaneous equations from the previous section. The Bellman equation did not tell us anything we did not already know, but it gives us a general procedure that works even when the optimal policy is not obvious by inspection.

Putting it together, starting from Writing and expanding two full rounds:

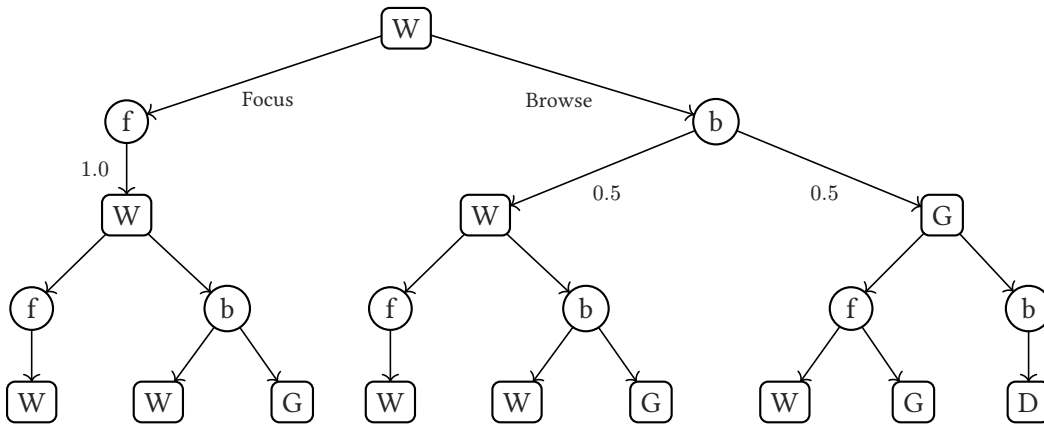


Figure 5: Depth-2 expectimax tree rooted at Writing. Square = agent node, circle = Q-state. W = Writing, G = Googling, D = DS. Probabilities on edges match Table 1; and they are omitted at depth 2 for clarity.